



C3

Token economics + the case

~50 min · one case, two legs: it works → can it pay?

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Put a number on what an AI answer costs, explain how each prompting technique moves that number, and judge whether a system that works is a system that can pay.

From “does it work?” to “can it pay?”

Every technique you learned in Capsule 2 has a price tag. Today we read the bill.

The uncomfortable question

You now know how to make a model answer better: more examples, more reasoning, more samples.

Every one of those is “send more tokens” or “generate more tokens” — on every single call, forever.

A system that is excellent and unaffordable is not a system. It is a demo.

- **Prompting cost is a per-call cost** — you pay it on call one and on call one billion — unlike training, which you pay once
- **So quality becomes a purchase** — “how accurate do we need this to be?” is a budget question, not only a technical one

Takeaway Accuracy is bought in tokens — so someone has to decide what the last percentage point is worth.

What you actually pay for

Four meters are running on every call. Most people only think about the first.

Meter	What it is	What makes it grow
Input tokens	everything you send: instruction, rules, examples, retrieved context	few-shot examples • long system prompts • big RAG chunks
Output tokens	everything the model writes back	chain-of-thought • verbose formats • long answers
“Thinking” tokens	reasoning models generate hidden steps you are still billed for	harder questions • higher reasoning effort settings
Retries & samples	the same question asked more than once	self-consistency (n×) • validation failures • fallbacks

Output tokens are typically priced several times higher than input tokens — so “make it explain itself” costs more than “give it more context”.

Takeaway You are billed for what you send AND what it writes — including thinking you never see.

The ladder from Capsule 2 — now with prices

Same six rungs. This is what each one does to the bill.

Rung	What it costs	Worth it when...
1 • Be clear & direct	nothing	always — try this first
2 • Few-shot examples	+input on EVERY call, forever	format/consistency failures; keep examples short
3 • Chain-of-thought	+output (the expensive meter), often several ×	genuine multi-step reasoning — not lookups
4 • Structured output	≈ nothing (often SAVES tokens — no prose padding)	almost always
5 • System / role prompt	+small input per call	tone/rules you would otherwise repeat
6 • Self-consistency	× n — you pay n times for one answer	being wrong is more expensive than being slow

Takeaway The cheapest rungs (1 and 4) are also the ones people skip. Reach for rung 6 last, and deliberately.

Under the hood — the only formula you need

FOR BUILDERS

Cost-to-serve, in one line.

$\text{cost} = (\text{input_tokens} \times \text{price_in} + \text{output_tokens} \times \text{price_out}) \times \text{calls} \times \text{samples}$

Everything else — caching, batching, a smaller model, a shorter prompt — is just making one of those five terms smaller.

Lever	Effect
Trim the prompt / shorter examples	↓ input_tokens on every call
Cap the output / ask for JSON not prose	↓ output_tokens
Route easy queries to a smaller model	↓ price_in and price_out
Cache repeated context	↓ effective input_tokens
Drop from n samples to 1	↓ samples (the biggest single lever)

Takeaway Five terms. Every cost optimisation you will ever read about is shrinking one of them.

Which change adds the MOST cost per answer?

A • Adding two few-shot examples

B • Asking it to think step by step

C • Forcing JSON output

D • Sampling 5 answers and voting

Think about which meter each one moves — and by how much.

Your own build, at scale

The v4 prompt you just watched — priced as if a bank actually shipped it.

	Per call	× 1,000,000 calls
Input (rules + 2 examples + the form) ≈ 400 tokens	~\$0.00006	~\$60
Output (the JSON) ≈ 80 tokens	~\$0.00005	~\$50
Total, single answer	~\$0.00011	~\$110
Same prompt with self-consistency n=5	~\$0.00055	~\$550

The judgement call

Is 10/10 instead of 9/10 worth ~\$440 more per million answers? Sometimes yes, sometimes no — that is the job.

Illustrative prices; your notebook prints your real token counts.

Takeaway Small numbers × big volumes = a real budget line. Always multiply out before you commit.

THE CASE • LEG A

Med-PaLM 2 — it works.

The paper you pre-read: how a general model was pushed to expert-level medical answers — and what the recipe was.

Leg A • The situation

CASE

Medical question answering, where a confident wrong answer can kill someone.

- **The task** — answer real medical questions — US Medical Licensing Examination (USMLE)-style multiple choice, and long-form questions from patients and clinicians
- **Why it is hard** — it needs recall, multi-step clinical reasoning, and safety; a fluent wrong answer is the worst possible output (your C1 think-pair-share, made real)
- **The starting point** — a general foundation model (PaLM 2) — not a model trained from scratch on medicine
- **The bar** — not "beat a benchmark" — be good enough that physicians would prefer your answer

Takeaway Same next-token engine you met in Capsule 1 — pointed at a domain where errors are unacceptable.

Leg A • The recipe — four techniques, stacked

Almost all of the gain came from how they PROMPTED it, not from a new model.

Technique	What it did	Rung (C2)
Instruction-prompt tuning	taught the model the medical answering style with a small tuned prompt	~2
Few-shot examples	worked medical examples in the prompt	2
Chain-of-thought	made it reason through the case step by step	3
Self-consistency	sampled many reasoning paths, took the majority answer	6
Ensemble refinement	generated several paths, then conditioned a FINAL answer on them	6+

Every one of these is a rung you used in Capsule 2 — the same ladder, in a hospital.

Takeaway No new model. A better prompt, stacked five ways.

Leg A • How they proved it

They refused to judge quality on one number. Nine named axes, judged blind by physicians.

~86.5% on MedQA (USMLE-style), +19 points — but that says nothing about whether a long answer is safe:

4 • HIGH-QUALITY TRAITS (more is better)	5 • POTENTIAL RISKS (less is better)
1 • Better reflects medical consensus	5 • Includes inaccurate or irrelevant information
2 • Better reading comprehension	6 • Omits important information
3 • Better knowledge recall	7 • Shows evidence of demographic bias
4 • Better reasoning	8 • Extent of possible harm
	9 • Likelihood of harm

Judged blind against physicians' own answers, Med-PaLM 2 won 8 of these 9.

The one it lost: axis 5 — its answers included inaccurate or irrelevant information more often than physicians' did. Fluent, confident, and padded — exactly the C1 failure mode.

Takeaway They built a better ruler before claiming a better answer — exactly what you did with your ten cases.

Which technique bought the most accuracy per token?

Rank the four: few-shot examples · chain-of-thought · self-consistency · ensemble refinement.

In your groups: rank them by VALUE FOR TOKENS, best first.

You have not seen any prices yet — argue from what each one does.

One line of reasoning per group. 5 minutes.

Deliberately before the prices — we compare your ranking to the bill in a moment.

THE HINGE

Now put that recipe on the meter.

The thing that made it expert-level is the thing that makes it expensive.

The recipe, multiplied out

Take a single plain answer as 1×. Now stack the recipe.

Add...	Effect on one answer	Running multiple
a plain answer	baseline	1×
+ few-shot medical examples	more input tokens on every call	~1.5×
+ chain-of-thought	several × the output tokens (the dearer meter)	~4–6×
+ self-consistency over n samples	the whole thing, n times	× n (n=5 → ~20–30×)
+ ensemble refinement	one more pass over those samples	more again

So the same question can cost 1× or 30×

— depending only on how you prompted it. Nothing about the model changed.

(Illustrative arithmetic — not figures from the paper.)

Takeaway Accuracy was bought — and the receipt is a 20–30× multiplier on every single answer.

THE CASE • LEG B

AI Wars in 2025 — can it pay?

June 2025. Alphabet has shed a quarter-trillion dollars in market value, and Google searches have fallen for the first time in twenty years.

CASE

Leg B • The situation

Google invented the technology that is now eating its business model.

- **The spark** — Google searches dipped for the first time in 20 years; hundreds of millions were asking chatbots — ChatGPT, Perplexity, DeepSeek — questions that used to go to Search
- **The irony** — Google's own scientists invented the transformer; Gemini is competitive; yet OpenAI defined the category
- **The engine at risk** — search advertising still prints hundreds of billions — but ads live beside a page of links, not inside a chat answer
- **The decision** — defend Search by turning it into a chatbot (new costs, unclear ad model) — or wait, and risk obsolescence

Takeaway The best technology in the room does not automatically become the best business.

Leg B • The unit economics of one search

Three numbers decide the entire question. All are per single search.

	US cents per search	Where it comes from
Revenue — advertising	1.61 ¢	what a search earns Google today
Cost — index search	1.06 ¢	running classic Search: crawl, index, rank
Cost — AI inference (basic)	0.356 ¢	added by putting generative AI in the answer

So the margin per search...

Today: $1.61 - 1.06 = 0.55 \text{ ¢}$

With basic AI: $1.61 - (1.06 + 0.356) = 0.19 \text{ ¢}$ — about a 65% cut

Across ~5 trillion searches a year, that gap is roughly \$18 billion of operating income.

Figures from the case (HBS 725-484) and its unit-economics courseware (726-851).

Takeaway Adding AI to a 1.61-cent product costs a third of a cent — and two-thirds of the margin.

Leg B • Four futures

THE EXERCISE

The courseware model, run four ways. Operating income per year, in US\$ billions.

Scenario	What changes	Op. income / yr
1 • Baseline, no AI	search as it is today	+\$27.5 bn
2 • Add AI, people chat (10× longer prompts)	inference × 10	−\$150 bn
3 • ...and wait 5 years (inference −30%/yr)	cost falls ~83%	−\$2.4 bn
4 • ...and charge \$20/month (400 M subscribers)	ads → subscription	+\$39.6 bn

Read the story, not the rows

Naive AI destroys the business. Falling costs alone only get you back to break-even.

A different business model — not a cheaper model — is what makes it work.

Takeaway The technology question was answered years ago. This is a business-model question.

If quality is a function of token spend — who wins?

1 • The best model, or the cheapest one that is good enough? Argue it.

2 • At what price per answer would you deploy a Med-PaLM-class system in a hospital?

Groups: pick ONE question. 8 minutes, then we share.

Connect both legs: Leg A bought accuracy with tokens; Leg B shows what tokens do to a P&L.

Your build — what would it cost your employer?

Take the v4 prompt from Capsule 2 (~400 in / 80 out tokens).

Estimate the calls per year for a real use case at your company.

Multiply it out. Now: would you turn on self-consistency ($\times 5$)?

5 minutes — then three teams share a number and a decision.

~\$110 per million answers, single-sample. $\times 5$ for voting.

What you would actually do about it

FOR BUILDERS

The levers a team reaches for when the bill arrives — in the order they usually try them.

Lever	What it does	Catch
Shorten the prompt	fewer input tokens on every call	may lose the accuracy you paid for
Cache the fixed prefix	stop re-paying for unchanged rules/examples	only helps if the prefix is stable
Route by difficulty	small model for easy queries, big one for the tail	you must classify reliably — and cheaply
Cap or structure the output	fewer output tokens (the dearer meter)	JSON already does much of this
Sample once, not n times	the single biggest saving	you give back reliability

Google's own options in the case are the same list at industrial scale — plus one more: change what you charge for.

Takeaway Every lever trades quality, latency or reliability for money. Choose knowingly.

CAPSULE 3 • TAKEAWAY

Accuracy is bought in tokens.

Decide what the last percentage point is worth — then design the prompt, the model and the business model to match.

What we learnt today — Class 3

RECAP

The whole session in six lines. Same shape as the recap we opened with.

- 1 **Inside the model** — tokens, attention and the transformer — it predicts the next token, and that is the whole engine
- 2 **Why it makes things up** — brilliance and hallucination come from the same mechanism; there is no built-in “I’m not sure”
- 3 **Prompting is a ladder** — six rungs — be clear • few-shot • chain-of-thought • structured output • role • self-consistency. Pick the rung to the failure
- 4 **Structure makes it testable** — JSON turns an answer into a component a program can use and a test can check
- 5 **Evals turn opinion into a number** — write the ruler first, change ONE thing, watch the pass-rate — “better, not just different”
- 6 **Accuracy is bought in tokens** — Med-PaLM’s recipe cost 20–30× per answer; AI Wars showed what that does to a 1.61¢ product

Throughline: You opened the model, learned to steer it, proved it worked — and then counted what it costs.

Coming in Class 4 — Agents & tool use

[NEXT SESSION](#)

Today one question was one call. Next week one question becomes a loop of calls.

- 1 **What is an agent** — a model + tools + a control loop; agent vs a plain prompt vs the RAG you built in Week 2
- 2 **Tool use & design** — function calling, designing good tools, and the “act” step that closes the loop
- 3 **Agent failure modes** — loops, cascading errors and cost blow-ups — plus how you evaluate an agent
- 4 **Agent economics** — a multi-step agent multiplies today’s token maths — the direct sequel to this capsule
- 5 **You’ll build** — a ReAct agent with one tool — the next step up from today’s structured-output prompt
- 6 **Case & milestone** — HBS case: “From LLM to Agent? Anthropic’s Next Move” • A1 due Sun 30 Aug 2026, 23:59

Throughline: Everything you learned today multiplies when a single request becomes many model calls.